```
package com.etat.mer

import kotlinx.coroutines.flow.MutableSharedFlow
import kotlinx.coroutines.flow.asSharedFlow

// Structure pour stocker la norme de l'accélération et l'état de la mer
data class EtatMerData(
    val norme: Double,
    val etatMer: String
)
object EtatMerEventBus {
    private val _etatMerFlow = MutableSharedFlow<EtatMerData>(replay = 1)
    val etatMerFlow = _etatMerFlow.asSharedFlow()
    fun diffuserEtatMer(data: EtatMerData) {
        _etatMerFlow.tryEmit(data)
    }
}
```

```
package com.etat.mer


class EtatMerService: Service(), SensorEventListener {

    private val TAG="EtatMerService"
    private lateinit var sensorManager: SensorManager
    private var accelerometer: Sensor? = null

    override fun onBind(p0: Intent?): IBinder? {
        return null
    }

    override fun onCreate() {
        super.onCreate()
        Log.i(TAG,"Service démarré...")
        sensorManager = getSystemService(SENSOR_SERVICE) as SensorManager
        accelerometer = sensorManager.getDefaultSensor(Sensor.TYPE_ACCELEROMETER)

        if(accelerometer!=null){
            sensorManager.registerListener(this, accelerometer, SensorManager.SENSOR_DELAY_NORMAL)
        }
    }

    override fun onDestroy() {
        sensorManager.unregisterListener(this)
        Log.i(TAG,"Service arrêté...")
        super.onDestroy()
    }



    override fun onSensorChanged(event: SensorEvent?) {
        if(event!=null) {
            if (event.sensor.type == Sensor.TYPE_ACCELEROMETER) {
                val x=event.values[0].toDouble()
                val y=event.values[1].toDouble()
                val z=event.values[2].toDouble()
                val norme=Math.sqrt(Math.pow(x,2.0)+Math.pow(y,2.0)+Math.pow(z,2.0))
                val etatMer=when{
                    norme<13 -> "Mer Calme"
                    norme<17 -> "Mer Agitée"
                    else -> "Mer Très Agitée"
                }
                Log.i(TAG,"Norme:$norme  =>  $etatMer")
            }
        }
    }
    override fun onAccuracyChanged(p0: Sensor?, p1: Int) {

    }
}
```

```
package com.etat.mer
class MainActivity : AppCompatActivity() {
    private lateinit var amb: ActivityMainBinding
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        initApp()
    }
    private fun initApp() {
        amb = ActivityMainBinding.inflate(layoutInflater)
        setContentView(amb.root)
        ajouterEcouteurs()
    }
    private fun ajouterEcouteurs() {
        lifecycleScope.launch {
            repeatOnLifecycle(Lifecycle.State.STARTED) {
                EtatMerEventBus.etatMerFlow.collect { data ->
                    amb.tvEtatMer.text = "${data.norme}  => ${data.etatMer}"
                    when{
                        data.norme<13-> amb.imgEtatMer.setImageResource(R.drawable.mer_calme)
                        data.norme<17-> {amb.imgEtatMer.setImageResource(R.drawable.mer_agitee)
                                attendre(5)}

                        else-> {amb.imgEtatMer.setImageResource(R.drawable.mer_tres_agitee)
                            attendre(5)}
                    }
                }
            }
        }
    }
    private fun attendre( timeSec:Long){
        lifecycleScope.launch {
            arreterService()
            // Attente non-bloquante de 5 secondes (5000 millisecondes)
            delay(timeSec*1000)
            // 2. Code après les 5 secondes (ex: masquer le chargement, changer d'écran)
            demarrerService()
        }
    }
    override fun onResume() {
        demarrerService()
        super.onResume()
    }
    override fun onPause() {
        arreterService()
        super.onPause()
    }    protected fun demarrerService() {
        val i = Intent(this, EtatMerService::class.java)
        startService(i)
    }
    protected fun arreterService() {
        val i = Intent(this, EtatMerService::class.java)
        stopService(i)
    }
}
```